Task Management API - Project Blueprint

Objective

Build a production-style REST API using:

- Node.js
- Express.js
- MongoDB
- Mongoose
- JWT Authentication

The project should demonstrate:

- Authentication
- Authorization
- CRUD Operations
- Search
- Filtering
- Pagination
- Sorting
- Validation
- Logging
- Swagger Documentation

Architecture

Client → Routes → Middleware → Controller → Service → Repository → MongoDB

Each layer has one responsibility.

Folder Structure

src/

config/

- db.js
- swagger.js

controller/

- auth.controller.js
- task.controller.js

service/

- auth.service.js
- task.service.js

repository/

- user.repository.js
- task.repository.js

middleware/

- auth.middleware.js
- role.middleware.js
- validation.middleware.js
- error.middleware.js

routes/

- auth.routes.js
- task.routes.js

models/

- User.js
- Task.js

validators/

- auth.validator.js
- task.validator.js

utils/

- ApiError.js
- ApiResponse.js
- logger.js

app.js server.js

Database Design

User

- name
- email
- password
- role

Task

- title
- description
- status
- priority
- createdBy

Roles

Admin

- View all tasks
- Update any task
- Delete any task

User

- Manage only own tasks

Build Phases

Phase 0 - Planning

Goal:

Understand architecture.

Learn:

- Routes
- Middleware
- Controller

- Service
- Repository

Deliverable:

Architecture diagram.

Phase 1 - Project Setup

Goal:

Create backend skeleton.

Tasks:

- npm init
- Express setup
- Environment variables
- MongoDB connection

Concepts:

- Express Application
- Environment Variables
- Mongoose Connection

Deliverable:

Server running successfully.

Phase 2 - User Model

Goal:

Store users.

Tasks:

- Create User Schema
- Create User Model

Concepts:

- Schema
- Model

Deliverable:

User model created.

Phase 3 - Authentication

Goal:

Signup/Login.

Tasks:

- Signup API
- Login API
- Password Hashing
- JWT Generation

Concepts:

- bcrypt
- JWT

Deliverable:

User can register and login.

Phase 4 - Authentication Middleware

Goal:

Protect routes.

Tasks:

- Verify JWT
- Attach user to request

Concepts:

- Middleware
- Authorization Header

Deliverable:

Protected routes working.

Phase 5 - Authorization

Goal:

Role-based access.

Tasks:

- Admin Middleware
- User Middleware

Concepts:

- RBAC

Deliverable:

Role restrictions working.

Phase 6 - Task Model

Goal:

Create task structure.

Tasks:

- Task Schema
- Relationship with User

Concepts:

- ObjectId References

Deliverable:

Task model created.

Phase 7 - CRUD APIs

Goal:

Task management.

Tasks:

- Create Task
- Get Task
- Update Task
- Delete Task

Concepts:

- REST APIs

Deliverable:

Basic CRUD completed.

Phase 8 - Service Layer

Goal:

Separate business logic.

Tasks:

- Move logic from controller

Concepts:

- Separation of Concerns

Deliverable:

Thin controllers.

Phase 9 - Repository Layer

Goal:

Separate database logic.

Tasks:

- Move Mongoose queries

Concepts:

- Repository Pattern

Deliverable:

Database operations isolated.

Phase 10 - Validation

Goal:

Prevent invalid data.

Tasks:

- Joi Schemas

Concepts:

- Request Validation

Deliverable:

Validation middleware.

Phase 11 - Filtering

Goal:

Query specific tasks.

Examples:

?status=pending

?priority=high

Deliverable:

Filtering working.

Phase 12 - Search

Goal:

Search tasks.

Examples:

?search=meeting

Concepts:

- Regex Search

Deliverable:

Search working.

Phase 13 - Pagination

Goal:

Split large results.

Examples:

?page=1&limit=10

Concepts:

- skip()
- limit()

Deliverable:

Pagination working.

Phase 14 - Sorting

Goal:

Sort records.

Examples:

?sort=-createdAt

Deliverable:

Sorting working.

Phase 15 - Logging

Goal:

Track requests and errors.

Tools:

- Morgan
- Winston

Deliverable:

Logs generated.

Phase 16 - Swagger

Goal:

Document APIs.

Tasks:

- Setup Swagger
- Document endpoints

Deliverable:

/api-docs working

Phase 17 - Testing

Goal:

Verify everything.

Tools:

- Postman

Deliverable:

Complete tested API.